

Data-Driven RSI: Evaluation, Ladders, and Trajectories

What RSI must first take over is the improvement loop now driven by foundation-model teams: find problems, design evaluations, produce data, train, and choose the next move. This note starts with the part of that loop that seems easiest to test—data.

Eval

Synthetic data

Harness-Evolve

What RSI is meant to take over

Many visions of RSI become the same engineering question: can models gradually take over the improvement loop that foundation-model teams still maintain by hand? Teams at OpenAI, Moonshot AI—the company behind Kimi—and Zhipu AI—the developer of GLM—still rely on researchers to find failures, build evaluations, produce data, run training, and choose the next direction.

Put bluntly, RSI is meant to take over work that foundation-model researchers like us still perform. Even without people continuously tending the loop, an LLM should be able to locate its capability boundary, propose worthwhile problems, create experience for the next training round, and test whether its successor is genuinely stronger.

That is closer to the form of RSI I care about than a model directly rewriting its own weights. The object being taken over is not one local optimization, but a research loop that can keep running.

Why start with data

This may not begin with a model rewriting its architecture. It is hard to predict where the next important algorithmic change will appear or whether an idea will survive scale. Data is more concrete: give a model unfamiliar tasks, locate stable failures, turn them into new tasks and trajectories, and train those back into the next model.

The point is not simply to feed the model more tokens. The point is that its failures change what gets produced next. If this loop can keep running, the model is already participating in the decision about what it should learn next. That remains short of full RSI, but it may be the first part that can be tested seriously.

Following that line of thought leaves me with three questions:

- How can an open-ended goal such as “be better at mathematics” or “be better at coding” be evaluated well enough to decide what to train next?
- When a batch of synthetic data looks good, how can its absorption by the target model be measured, and how much should be produced?
- How can synthesis techniques accumulated across domains become a repeatable production line for useful trajectories?

Those are the three questions considered here.

Working definitions

RSI here does not mean that a model rewrites its parameters during a single inference run. I use a weaker, testable definition: the current model helps identify capability gaps and produce experience for the next training round; the resulting successor then improves reproducibly on fresh tasks that were not part of production.

Open-ended evaluation does not mean subjective evaluation. The goal may remain broad—for example, mathematical research ability—but each iteration must leave auditable evidence: tasks, trajectories, failure causes, verifier outcomes, and a data prescription for the next round.

Harness–Evolve is a working name used in this note, not an established algorithm. The harness defines the environment in which the model operates. Evolve generates, revises, and selects candidate trajectories inside that environment. SFT then trains selected processes into the target model. Evolve itself does not update the weights.

Why this leads to synthetic data

Push the data-driven view one step further and synthetic data becomes unavoidable.

Many SFT examples, preference pairs, critiques and rewrites, verifier-filtered rollouts, and tool-use trajectories are not simply collected from the open web. They are constructed through some combination of models, rules, tools, and people. Many recent post-training gains rely on this deliberately constructed experience.

Pretraining is beginning to acquire the same character. Natural data remains the foundation, but once cleaning, deduplication, and filtering improve, more of the marginal data will come from rewriting, transformation, difficulty control, reasoning completion, execution-based filtering, or turning existing material into a task that is actually worth learning. The boundary between pretraining and post-training data will become less clean over time.

This does not make natural data unimportant. Natural data gives a model its knowledge, language, and contact with the world. But the internet does not grow in response to the failure modes of a particular checkpoint. When a model gets stuck on one step of one kind of problem, the web does not immediately produce ten thousand exercises at exactly the right difficulty with reliable feedback attached.

The data wall does not mean that the internet has run out of new material. It means that data which is new to the model, useful for the next version, and good enough to train on is not growing as quickly as the appetite created by more models and more compute. Estimates of public text stocks and training demand suggest that high-quality human-generated text can become a constraint on continued scaling ([Villalobos et al., 2024](#)). This is not a precise date for impact, but it is enough to separate new tokens from genuinely new capability.

Will DePue calls the internet a “one-time subsidy” to deep learning ([DePue, 2026](#)). The phrase is useful. For decades, people wrote webpages, code, papers, and discussions for reasons unrelated to model training; together they happened to form a huge, inexpensive, cross-domain dataset. The next increment will not arrive through the same accident. What remains outside the public web is often not another collection of documents, but internal workflows, tacit expert judgment, unrecorded failures, and processes visible only inside real environments.

This separates the data problem into at least two axes. **Volume** asks how many more high-quality examples are needed in domains already represented. **Coverage** asks which tasks, tools, edge cases, and long-horizon processes have never been recorded at all. Synthetic data is well suited to expanding volume, but it cannot reconstruct information that does not exist in its inputs. Extending coverage still requires anchors from people and real environments, after which models can elaborate, recombine, and explore around them.

The scarce resource is not tokens. It is experience that is useful for the model's next step.

If scaling is to continue, it cannot depend on people writing a larger internet. Models will have to help create the next round of experience: turn failures into tasks, turn tool and verifier feedback into process, and turn the useful parts of that process back into training data. Synthetic data therefore becomes an important precondition for RSI. Of course, synthetic data can also copy the past more quickly. A model can generate millions of answers that contain nothing it did not already know. If the same model produces, judges, and filters those answers, its preferences can become more concentrated with every round. This is why evaluation, a data ladder, and a serious trajectory pipeline have to be designed together.

1. Evaluation cannot have an endpoint

The resolution of a fixed benchmark rarely lasts indefinitely. As frontier models approach its ceiling, adapt repeatedly to its task format, or encounter its examples during training, its ability to distinguish the strongest systems can decline. Different benchmarks, however, saturate at different rates and for different reasons. In a study of 60 widely used text benchmarks, nearly half showed high or very high saturation ([Akhtar et al., 2026](#)). Contamination makes the score harder to interpret still, with effects that vary by model and benchmark ([Singh et al., 2024](#)).

The usual response is to ask people to build a harder benchmark, but that process is becoming expensive. Humanity's Last Exam selected 2,500 questions from more than 70,000 submissions, involving nearly a thousand experts and several stages of review ([Center for AI Safety & Scale AI, 2025](#)). This does not prove a speed law, but it motivates a prediction I take seriously: if model iteration keeps accelerating, the human cycle for defining tasks, writing questions, maintaining answers, and refreshing validation sets will eventually fall behind the model's own improvement cycle.

Evaluation therefore cannot remain a periodically updated question bank. It has to enter the self-improvement loop. The goal can stay open-ended—better mathematical research, stronger coding, longer-horizon work—while the model uses the previous failure map to search for its next boundary, propose tasks, construct counterexamples, control difficulty, and continuously grow new benchmarks and validation sets. AutoBench already formulates benchmark creation as an LM-driven search for datasets that are salient, novel, and difficult ([Li et al., 2025](#)).

A model can help propose benchmark items, tests, and verifiers, but that does not let it prove its own improvement. An active benchmark used for diagnosis and training can keep changing. Private validation used to confirm progress should remain isolated from the training pipeline and hidden from the evaluated checkpoint, with task validity, answer correctness, and leakage checked through independent generators, environment evidence, or human review. Freezing the checkpoint is useful, but it is not sufficient on its own.

A model can write its next test. It cannot use its own opinion of that test as proof that it improved.

Self-evaluation here is not self-scoring. It means maintaining a moving capability boundary. Self-validation does not remove external evidence; it lets the model help construct a test whose result is then settled by evidence it cannot anticipate or manipulate.

From an open-ended goal to the next training round

A benchmark becomes a process for continually locating the boundary, not a static file.

01

Name the direction

Mathematics, coding, research, or longer-horizon work.

02

Search the boundary

Propose fresh tasks, counterexamples, and harder variants.

03

Build a failure map

Locate repeatable failures instead of keeping only a score.

04

Produce data and train

Turn failures into tasks, trajectories, and a successor model.

05

Validate the successor

Compare it on hidden tasks and independent evidence, then feed the result into the next boundary search.

VALIDATION BOUNDARY

Private validation

Hidden from the evaluated checkpoint and strictly separated from training.

Independent evidence

Execution, environment feedback, traceable sources, separate verifiers, and human audits.

Figure 1. Steps 01–05 form one iteration. Validation feeds the next boundary search, while the validation boundary prevents the system from writing the test, training on it, and declaring victory.

The output of an evaluation round should not be a score but a **failure record** that changes what happens next: the task slice, terminal error, earliest causal deviation in the trajectory, candidate mechanism, and proposed data intervention. Two runs can both end in a timeout while failing for entirely different reasons—bad planning, a broken tool call, or context-management failure.

I would ask for five layers of evidence:

1. **Outcome:** Was the task completed under an executable or explicit verifier?
2. **Process:** Where did the first causal deviation occur?
3. **Slice:** Does the mechanism repeat across a fresh task family rather than one bad case?
4. **Prescription:** Does the failure imply a concrete task, feedback, or harness intervention?
5. **Transfer:** After training, does the gain appear on private validation that did not participate in data production?

2. Ladders: use small experiments to decide whether to climb

Imagine that a coding harness can produce one million trajectories in a week. Should it run at full scale? The answer should not be a matter of taste. Synthetic data can look polished and be fully correct while doing almost nothing when trained into the target model.

A ladder is not a formula, nor is it a scaling law assumed to exist. It is a **stage-gated experiment**: break one expensive run into increasingly costly rungs; scale one object at a time while holding the other conditions as stable as possible; climb only when the signal over a control is repeatable. Its primary purpose is to impose discipline on both experimentation and scaling decisions.

In architecture scaling, each rung represents a larger compute budget. At every rung, the candidate should be compared with a matched baseline under a preregistered data distribution, objective, evaluation protocol, and rule for allocating parameters and tokens. The question is: **does the advantage over baseline survive scale and earn the next compute budget?**

A data ladder changes the object being scaled. Hold the target model, training configuration, and evaluation fixed; increase the amount or coverage of data from a fixed source; measure what each additional tranche contributes on held-out tasks. It asks: **how much new learning signal remains, and where does marginal return disappear?**

Synthetic data is harder because the dataset is an output of a production recipe. The producer, harness, verifier, sampling policy, and filter jointly determine its distribution; the target model determines whether those trajectories can be absorbed. A comparable synthetic-data ladder must therefore hold the **production recipe and target model fixed**, varying only the volume of accepted, deduplicated trajectories. If any part of the production recipe—the producer, harness, verifier, sampling policy, or filter—or the target model changes, the new point should not be appended to the old curve; the ladder should restart from its cheaper rungs.

Scaling production can itself change the output: acceptance may fall, duplication may rise, and easy cases may crowd out the long tail. The same trajectories can teach one target model a planning habit and another little more than wording. SynthLLM likewise reports saturation regions that vary with target-model scale (Qin et al., 2025). How much a system can generate is not how much it should generate for this model.

All three ladders make small experiments precede large ones, but their experimental units and failure conditions differ:

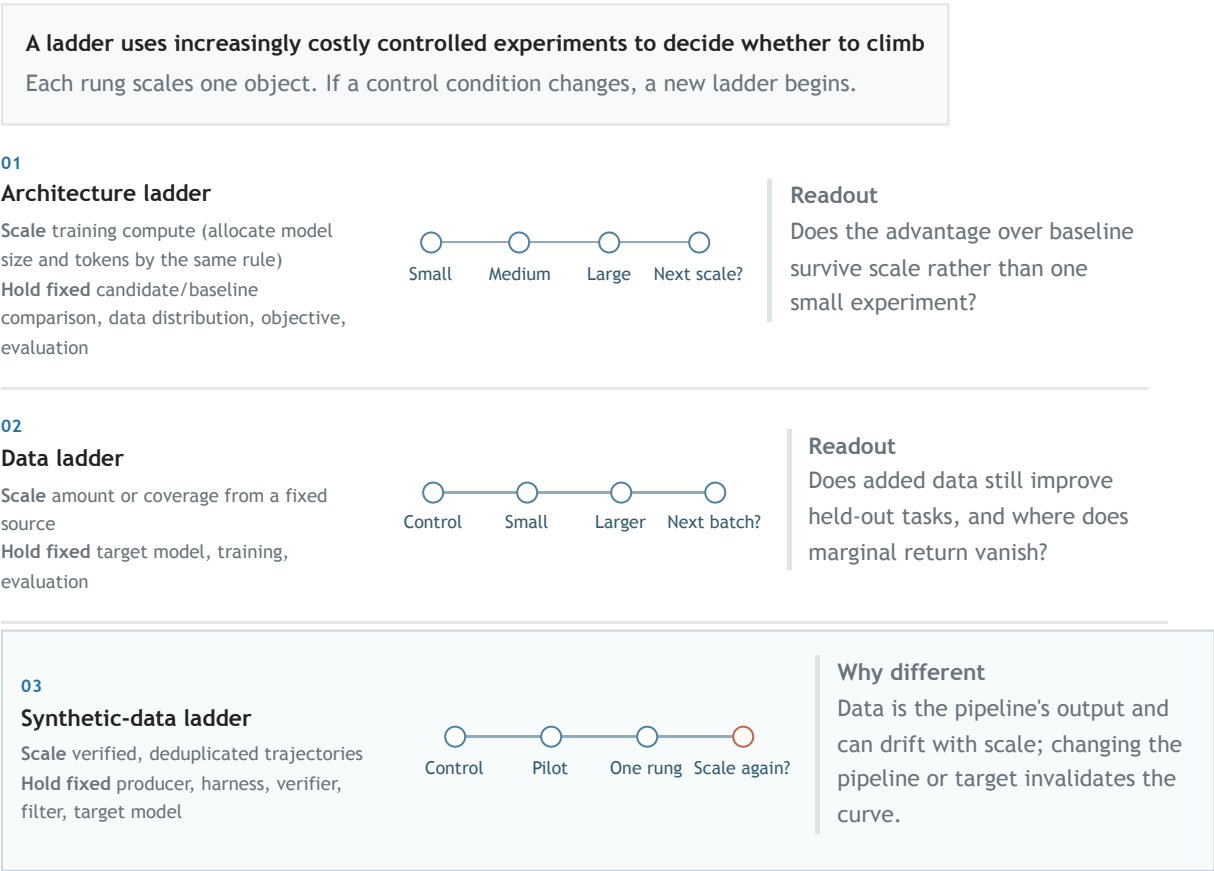


Figure 2. A synthetic-data ladder is not universal. It is a conditional curve for one production recipe and one target model.

A synthetic-data ladder is therefore a sequence of increasingly expensive bets: audit a small set of tasks and verifiers; run a pilot from the same base checkpoint; scale only after repeatable held-out signal; repeat when gain approaches the threshold implied by noise, cost, and regression risk; stop—or redesign the pipeline and begin a new ladder—when it falls below.

A minimum viable experiment

For the ladder to be interpretable, only a small number of variables should move at once. A minimum design fixes the base checkpoint, optimizer, number of epochs—or another preregistered exposure rule—and held-out evaluation, while varying only the number of accepted, deduplicated trajectories; total training tokens then grow with data volume. If the training-token budget is held fixed instead, the experiment measures coverage or mixture under equal compute, not pure data scaling, and should be reported separately. Each rung should be repeated at least twice so that training noise is not mistaken for a curve.

Each point needs three groups of measurements: generation cost, acceptance rate, and deduplication rate on the production side; effective training tokens, stability, and behavioral change on the training side; target-slice gain, cross-slice transfer, and regressions on the evaluation side. A more expensive rung is justified only when adjacent rungs show a consistent direction across repeats.

Strictly speaking, this remains a **ladder experiment**, not an extrapolatable scaling law, until it has been tested across enough scale points, target models, and production pipelines. Its first purpose is to pace investment and reveal saturation, not to promise a clean power law.

At every rung, ask again: how much new capability did the next batch buy?

The threshold combines uncertainty across repeats, production cost, and regression risk; it is not one universal score.

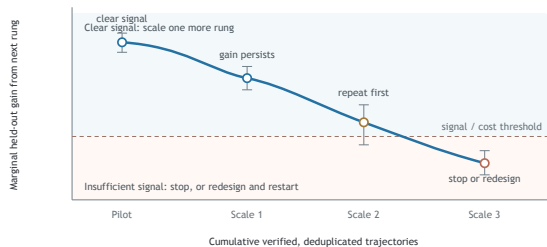


Figure 3. This is a decision rule, not an empirical result. Train every point from the same base checkpoint and repeat it on held-out tasks outside data production; do not extrapolate the curve to a new production recipe or target model.

What to measure first

The total number of generated examples is not very informative by itself. More useful measures are the share that passes verification, what remains after deduplication, and whether each additional batch of genuinely different trajectories improves an independent evaluation. The first two numbers determine what the production line really costs. The last one tells whether it should keep running.

Once a small training run shows a signal, accepted and non-duplicate trajectories can be scaled in stages. At each stage, the starting checkpoint, training setup, and evaluation set should remain as stable as possible, with the previous rung retained as a control. If volume grows several times while unfamiliar tasks no longer improve, or if acceptance collapses, the pipeline is approaching saturation.

The right amount of synthetic data is not the largest amount the budget can buy. It is the useful region before saturation. Without a ladder, it is easy to finish producing an expensive dataset and only then begin explaining why it did not work.

The ladder belongs to its pipeline and target model

The same synthetic dataset can behave very differently on two models. Every example carries habits from its generator and harness: how problems are decomposed, which tools are preferred, when the model backtracks, and which errors it tends to make. There is no guarantee that another model can absorb those habits.

Whenever the target model changes—or the producer, harness, verifier, sampling policy, or filter is modified—the ladder should return to its cheaper rungs. The previous curve is useful evidence, not a guarantee. This is one reason a scaling law for synthetic data is harder to establish than one for natural data.

3. Trajectories need a production line

Data has always been designed by people. People decide where to look, what is worth keeping, how to label it, and which tasks should come before others. Even when the raw material comes from the internet, it enters a model through a human-built pipeline.

As public webpages stop being the main source of marginal data, production will increasingly shift from collecting documents to recording work. A complete expert workflow can include private tools, repeated communication, local judgment, failure recovery, and final acceptance. Saving only the input and final answer discards much of the trainable process. Data infrastructure therefore means more than storage or labeling capacity; it must capture environments, actions, feedback, and outcomes together.

Many synthetic-data pipelines today are essentially prompt + model + filter. That is enough to produce large collections of questions and answers, but it remains too thin. The valuable part is often not the final answer. It is how the model searched, tried something, observed an error, revised a plan, and noticed that it had gone wrong.

A prompt is not a production line

In coding, people know that tests matter. In mathematics, they use proof checks, counterexamples, and controlled increases in difficulty. In agent tasks, environment state and tool output cannot simply be discarded. These pieces of experience currently live in prompts, scripts, and researchers' heads. A harness turns them into an environment the model encounters every time it works.

Here, a harness is more than an engineering wrapper. A fuller definition includes prompts, tools, context management, workflow, persistent state, permissions, and verification logic: the system that controls how a model observes, acts, stores artifacts, and checks itself (Weng, 2026). In practice, it determines the kind of trajectory the pipeline is able to collect.

Traceable

The task, generator, harness, tools, and verifier all have explicit versions.

Process is visible

Keep the necessary states, actions, tool feedback, failures, and revisions.

Verifiable

Prefer execution or independent judgment, and retain a reason when a trajectory is rejected.

Diverse

Control the mixture of harnesses and generators; remove duplicate paths and repeated phrasing.

Trainable and testable

Send every batch back through the ladder and an independent evaluation.

Harness–Evolve first, then SFT

Harness–Evolve can be read straightforwardly. Put a model in an environment with tools and feedback, and let it make several attempts. It can follow different paths, fail, backtrack, and revise in response to what happens. From those attempts, keep the complete processes that are correct, representative, and meaningfully different from one another.

What goes into SFT is no longer a polished final answer. It is a better trajectory. The aim is not to memorize one solution, but to make reliable behavior easier to reach the next time: verify when verification is needed, revise after failure, and use a tool when the task calls for one.

There is a clear line from STaR, which generates rationales, keeps paths that recover the correct answer, and trains them back into the model (Zelikman et al., 2022). Harness–Evolve expands the candidate object from a textual rationale to a stateful agent trajectory: tool responses, file changes, execution errors, backtracking, and replanning become part of the data. It is also different from DGM and Self-Harness, whose primary optimization target is the agent or harness itself. Here the direct output is training data, and the target model changes only in the final SFT stage (Zhang et al., 2025; Zhang et al., 2026).



Figure 4. These are four stages of one data pipeline. Evolve improves the candidate trajectories; it does not update the model directly. The final SFT stage writes the selected processes back into the target model.

Why use more than one harness

One harness tends to grow one style of trajectory. A test-driven coding harness produces many run–error–revision paths. A critique-and-rewrite harness produces more self-checking. A search-oriented harness produces branches and comparisons. Each can be useful, and each can harden into a template when overused.

I would rather run several harnesses and use the ladder to see which ones the target model actually absorbs, and which mixtures work better together. This is more cumbersome than declaring one ideal trajectory format, but it is less likely to train the model into a single repeated pattern.

A harness has a horizon too

There is a less visible constraint: harness code written for today’s agents may not support the long-running tasks that RSI eventually requires. Many systems assume that a task ends within one run or context, tool calls are synchronous, verifier feedback arrives quickly, and success can be represented by one terminal state. Those assumptions may be adequate for a coding task measured in minutes. They are less plausible for experiments that run for days, data programs spanning multiple training rounds, or research whose feedback arrives much later.

METR measures a task-completion horizon using the time a human expert needs for the same task. This is a proxy for task difficulty, not the agent’s literal wall-clock runtime. Their results nevertheless isolate an important capability axis: whether a model can reliably compose local skills into a longer sequence of actions (METR, 2026). If task horizon grows faster than harness horizon, the binding constraint may move away from the base model and into lost state, context growth, compounding errors, and failed recovery.

An RSI-oriented harness needs several capabilities that are not yet universal defaults:

- **Persistent state:** experiments, code, data versions, and unfinished work cannot live only in context; interrupted runs need recoverable checkpoints.
- **Hierarchical goals:** long tasks need verifiable stages while preserving dependencies across stages, rather than optimizing the nearest local score.
- **Asynchronous execution:** training, evaluation, and data generation may run for hours or days; the harness must launch, monitor, cancel, and resume background work.
- **Delayed feedback:** when final reward arrives late, intermediate evidence must be retained and failure attributed to the step that caused it.
- **Evolvability with boundaries:** the model may revise workflows, context policy, and tool composition, while verifiers, permissions, and audit logs remain outside the editable loop.

Recent work on long-horizon agents similarly centers compact state, checkpointing, verifier-backed state transitions, and targeted recovery rather than repeatedly feeding raw interaction history back into the prompt (Wu et al., 2026). This suggests a longer-term extension of the argument: Harness–Evolve cannot only evolve trajectories inside a fixed harness. As tasks lengthen, **the harness carrying those trajectories must evolve as well**. Otherwise the production line will reliably generate data only within its current horizon.

Synthesis still needs priors

Goal priors	Know roughly what “better at mathematics” contains without reducing it to one frozen test.
Task priors	Know which tasks are worth producing, how difficulty should increase, and which failures are informative.
Verification priors	Execute what can be executed and check what can be checked; avoid asking the generator to grade itself.
Exploration priors	Allow trial, error, and backtracking while cutting off long trajectories that are going nowhere.
Diversity priors	Control task types, tools, solution paths, and generator sources so one pattern does not fill the dataset.

These priors are not meant to write the solution for the model. They point to where exploration is useful, which feedback is trustworthy, and which outcomes should not be trusted. Ultimately, the model needs to learn more than a single synthesis recipe. Given a new topic or a new open-ended goal, it should be able to borrow from these experiences, design tasks and environments, and search for the next useful data.

There is also a practical risk: generators copy themselves. If one model acts, judges, and filters, Evolve can make its existing patterns more concentrated rather than more capable. Execution tools, independent verifiers, generators from different model families, anchors in real data, and small human audits are ways of keeping the production line from becoming an echo chamber.

A second risk is **selection-induced shortcutting**. A verifier that only checks the final answer may select lucky short paths rather than transferable processes; a preference for long traces can reward empty elaboration. Selection should therefore consider outcome, process completeness, trajectory novelty, and cost together, while retaining rejected traces and rejection reasons. Those records are useful not only for cleaning but for the next round of task and harness design.

Relationship to adjacent work

The nearest lines of work become easier to separate by their optimization target:

- **Reasoning self-training** optimizes reasoning paths to be internalized by the model. STaR is the clearest example.
- **Synthetic-data scaling** measures the gain and saturation of a generation method as data and model scale change. SynthLLM belongs here.
- **Dynamic evaluation** turns benchmark creation into continual search and refresh rather than maintenance of a permanent question bank. AutoBench is one example.
- **Harness optimization** changes prompts, tools, context, or workflow so that a frozen model performs better at deployment. Self-Harness closes a loop among failure mining, editing, and regression testing.
- **Open-ended agent evolution** retains a population of candidate agents and continues branching under executable feedback. DGM and AlphaEvolve show how archives and evaluators support this search ([Novikov et al., 2025](#)).
- **Harness–Evolve in this note** sits at their intersection: use a harness to expand trajectory search, use verifiers and diversity rules to select trajectories, and distill the result into a target model. It is a research hypothesis, not an empirical result.

The relevant tests are causal, not rhetorical: whether multiple harnesses produce more transferable data than one; whether a ladder predicts the return of large-scale production; and whether SFT learns a reusable process rather than the surface preferences of the generator and verifier.

Putting the loop together

Once the three pieces are connected, the workflow is ordinary: evaluation locates a problem, Harness–Evolve produces trajectories, SFT trains them back into the model, and the ladder decides whether that data method is still worth scaling.

The unusual part is that the failures of one model begin to determine the data for the next. As the model improves, old tasks stop being useful and old harnesses saturate. The system has to change its tasks, environments, and data rather than finishing one dataset and treating it as permanent.

01

Name the direction

The goal need not begin as one exact score, but it should be clear enough to explore.

02

Build fresh tasks

Keep changing the test until it reaches failures that matter for the current model.

03

Design the harness

Put human domain knowledge into tools, feedback, verification, and exploration rules.

04

Produce trajectories

Let models try, fail, and revise; retain several kinds of complete process.

05

Filter, then SFT

Train only trajectories that are reliable and meaningfully different.

06

Run the ladder

Measure absorption, transfer, and saturation before deciding to produce more.

To me, the strict definition of RSI matters less than whether the loop works. If a model can help identify its failures, help produce the next training experience, and become stronger because of that experience, the loop is already worth studying.

It is not autonomous self-improvement. The goal, harness, verifier, and stopping rule still depend heavily on human judgment. That is partly why the direction feels credible to me: one segment of the larger problem can be made measurable, scalable, and falsifiable without waiting for every other segment to be solved.

Open questions

- How can an open-ended goal be covered well enough without quietly replacing it with another benchmark?
- How can model-generated benchmarks and validation sets remain novel without mistaking the generator's preferences for the capability boundary?
- How should trajectories from different harnesses be deduplicated and mixed without cancelling one another out?
- When should the data keep scaling, and when should the generator or harness be replaced instead?
- Can the model gradually participate in harness design without causing evaluation and production to collapse into the same loop?
- Does SFT learn a better process, or merely a subtler version of the generator's style?

I do not have confident answers to these questions. The most useful next step may not be a more complete diagram. It may be to choose one narrow domain, run the entire loop several times, and see where it actually breaks.

References

1. Villalobos et al. (2024), [Will we run out of data? Limits of LLM scaling based on human-generated data.](#)

2. Singh et al. (2024), *Evaluation data contamination in LLMs: how do we measure it and (when) does it matter?*.
3. Zelikman et al. (2022), *STaR: Bootstrapping Reasoning With Reasoning*.
4. Qin et al. (2025), *Scaling Laws of Synthetic Data for Language Models*.
5. Zhang et al. (2025), *Darwin Gödel Machine: Open-Ended Evolution of Self-Improving Agents*.
6. Novikov et al. (2025), *AlphaEvolve: A coding agent for scientific and algorithmic discovery*.
7. Zhang et al. (2026), *Self-Harness: Harnesses That Improve Themselves*.
8. Weng (2026), *Harness Engineering for Self-Improvement*.
9. DePue (2026), *A Stargate for Data*.
10. METR (2026), *Task-Completion Time Horizons of Frontier AI Models*.
11. Wu et al. (2026), *StructAgent: Harness Long-horizon Digital Agents with Unified Causal Structure*.
12. Akhtar et al. (2026), *When AI Benchmarks Plateau*.
13. Center for AI Safety & Scale AI (2025), *Humanity's Last Exam*.
14. Li et al. (2025), *AutoBench: Creating Salient, Novel, Difficult Datasets for Language Models*.